

Lab 2: ROS Perception System Basics — Image Subscription and Processing

1. Experiment Objectives

- Learn how to subscribe to and process image data (RGB and Depth) in ROS.
- Master basic image processing techniques using OpenCV: Color Space Conversion (HSV), Object Detection, and Contour Detection.
- Understand how to calculate 3D coordinates of objects using Depth maps and Camera Intrinsics.
- Learn to transform coordinates from the Camera Frame to the Robot Base Frame using TF.
- Implement a basic Visual Servoing control loop to follow a target.
- Visualize 3D Point Clouds using Open3D.

2. Setup and Compilation

First, ensure your environment is set up and the package is compiled.

```
# 1. Navigate to your workspace
cd ~/catkin_ws

# 2. Build the package
# we whitelist only this package to speed up compilation
catkin_make -DCATKIN_WHITELIST_PACKAGES="lab2_perception"

# 3. Source the setup script to register the new package
source devel/setup.bash

# 4. Grant execution permissions
chmod +x ~/catkin_ws/src/lab2_perception/tools/*.py
chmod +x ~/catkin_ws/src/lab2_perception/scripts/*.py
```

3. Step-by-Step Experiment

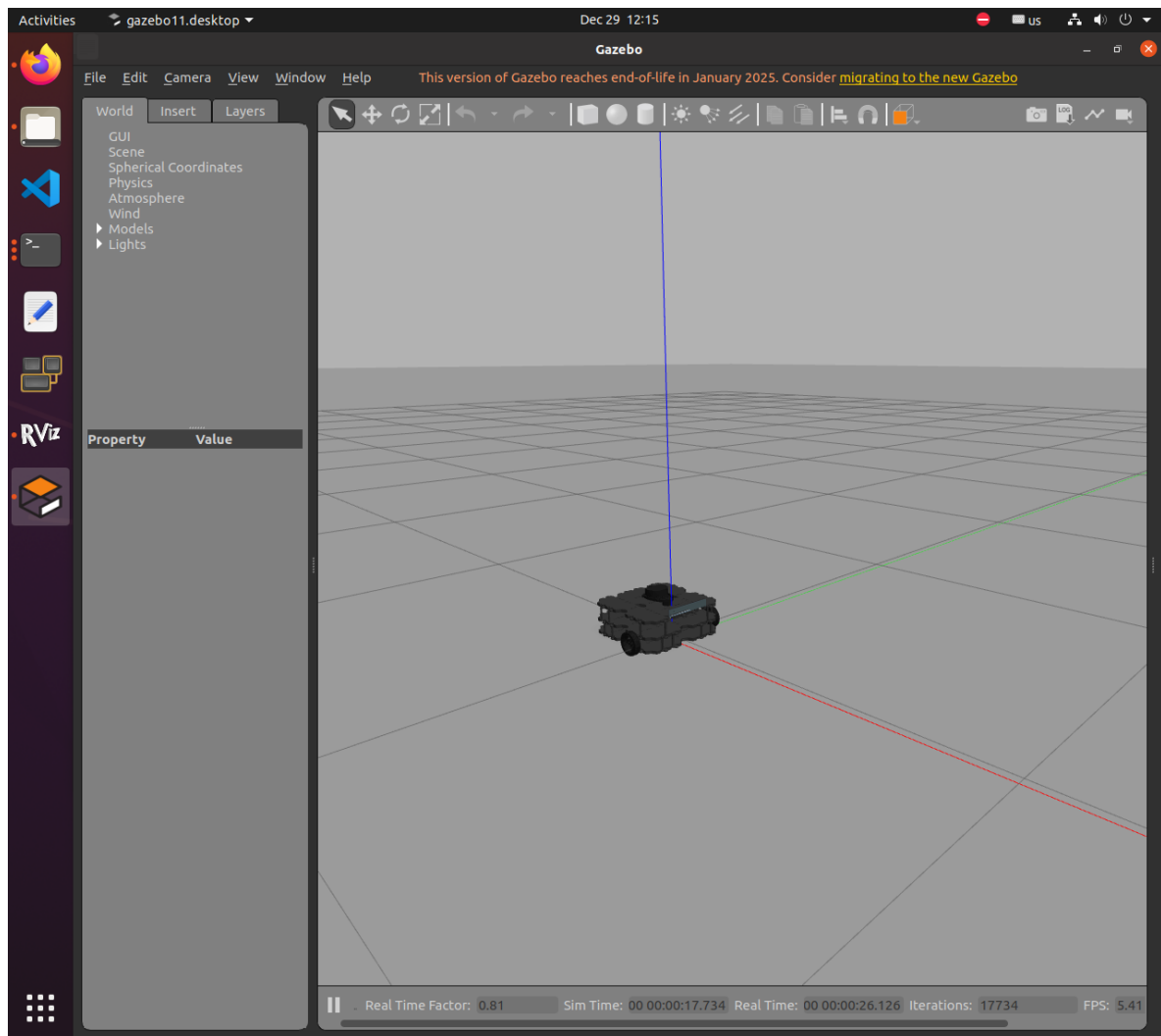
Part 1: Start simulation

Before diving into complex processing, let's verify we can receive images from the camera.

1.1 Run Simulation

```
# Terminal 1: Start ROS Master (if not running)
roscore
```

```
# Terminal 2: Start a robot simulation
source ~/catkin_ws/devel/setup.bash
export TURTLEBOT3_MODEL=waffle
roslaunch turtlebot3_gazebo turtlebot3_empty_world.launch
```



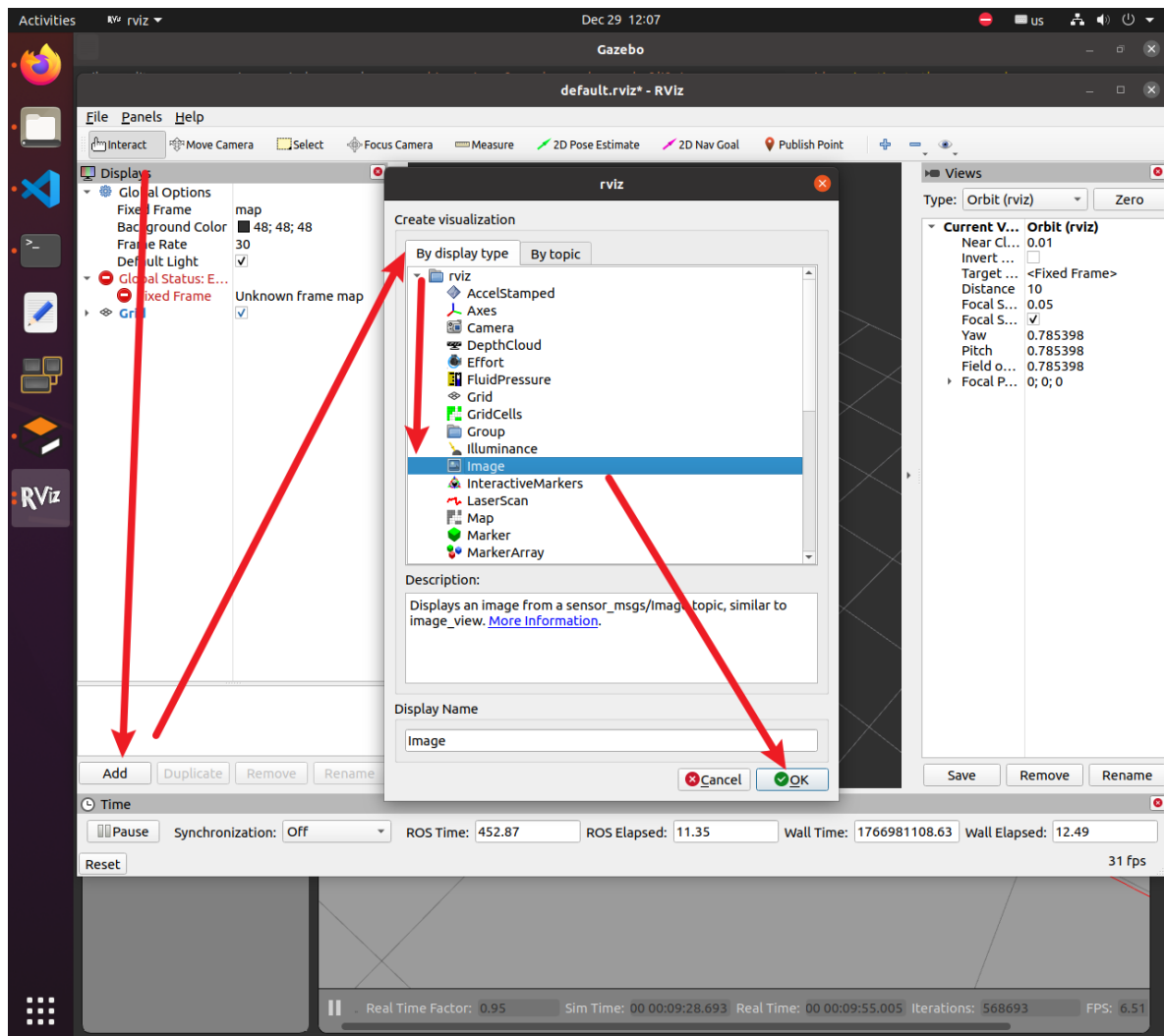
1.2 Run rviz to visualize rgb and depth image

Open RViz and add image displays for both RGB and depth topics.

```
# Terminal 3: Run RViz  
rviz
```

In RViz:

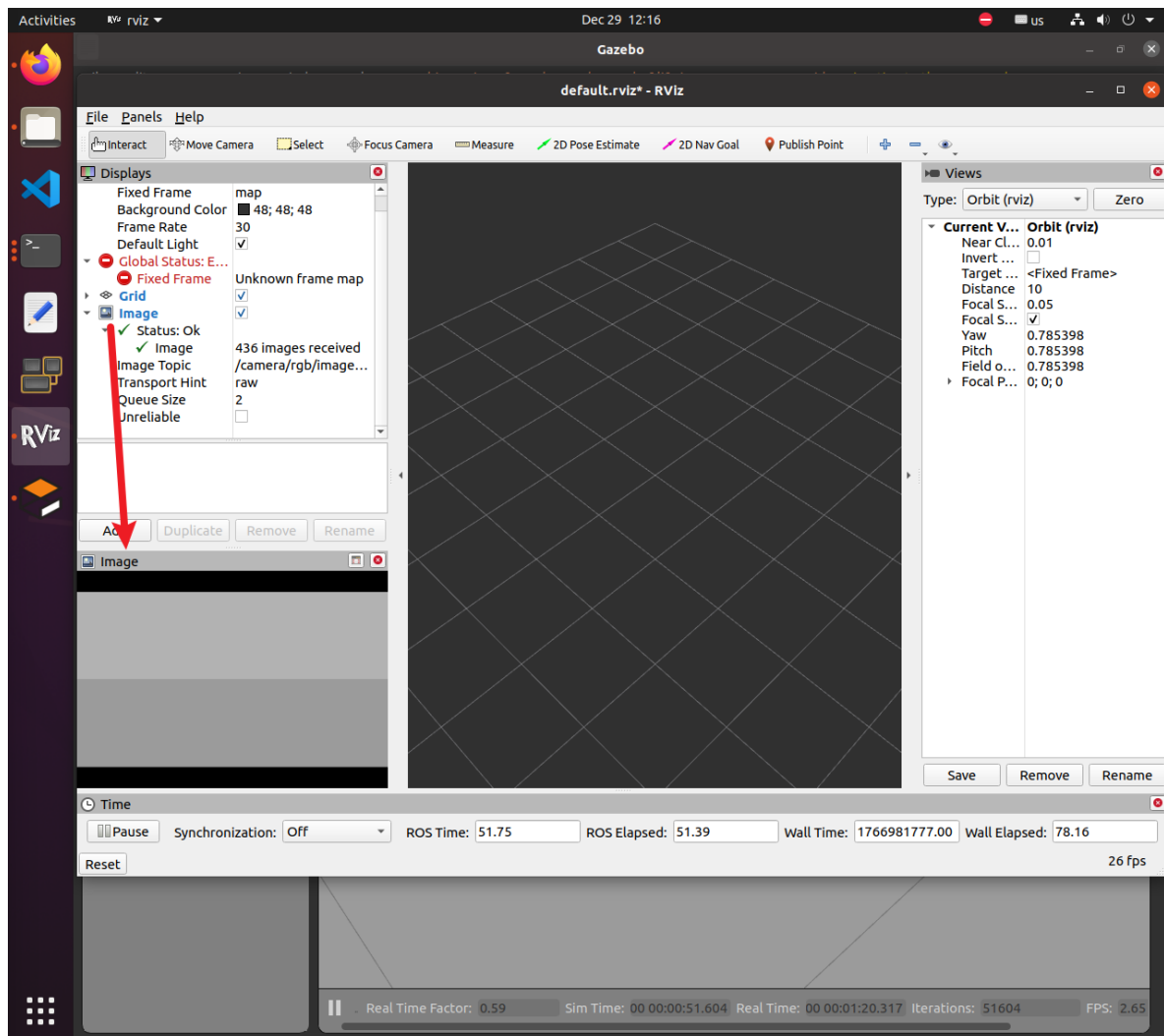
Click Add → By display type → Image



Use `rostopic list` to check the camera topic name.

```
s@ubuntu: ~/catkin_ws
s@u... x  rosc... x  /hom... x  s@ub... x  s@ub... x  s@ub... x
s@ubuntu:~/catkin_ws$ rostopic list
/camera/depth/image_raw
/camera/rgb/image_raw
/camera/rgb/image_raw/mouse_click
/clicked_point
/clock
/cmd_vel
/gazebo/link_states
/gazebo/model_states
/gazebo/parameter_descriptions
/gazebo/parameter_updates
/gazebo/performance_metrics
/gazebo/set_link_state
/gazebo/set_model_state
/imu
/initialpose
/joint_states
/move_base_simple/goal
/odom
/rosout
/rosout_agg
/scan
/tf
/tf_static
```

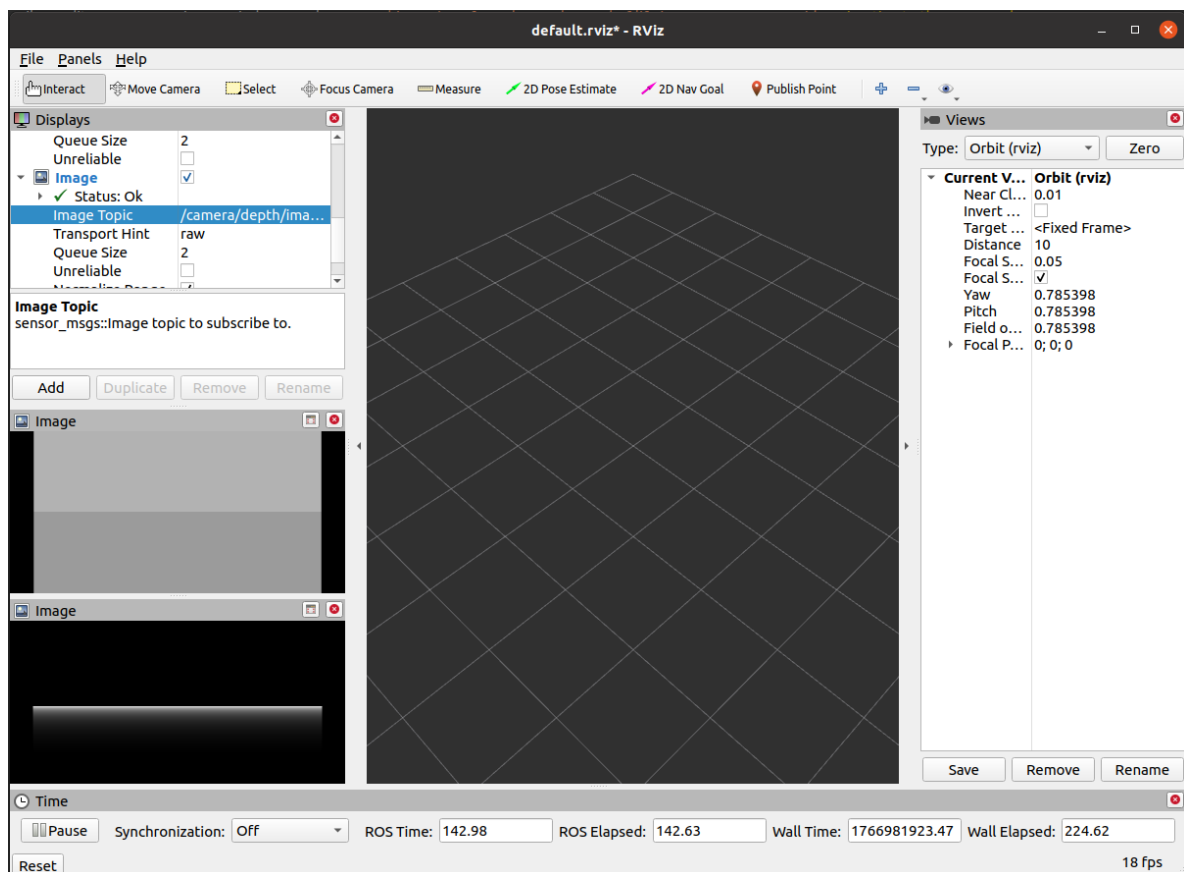
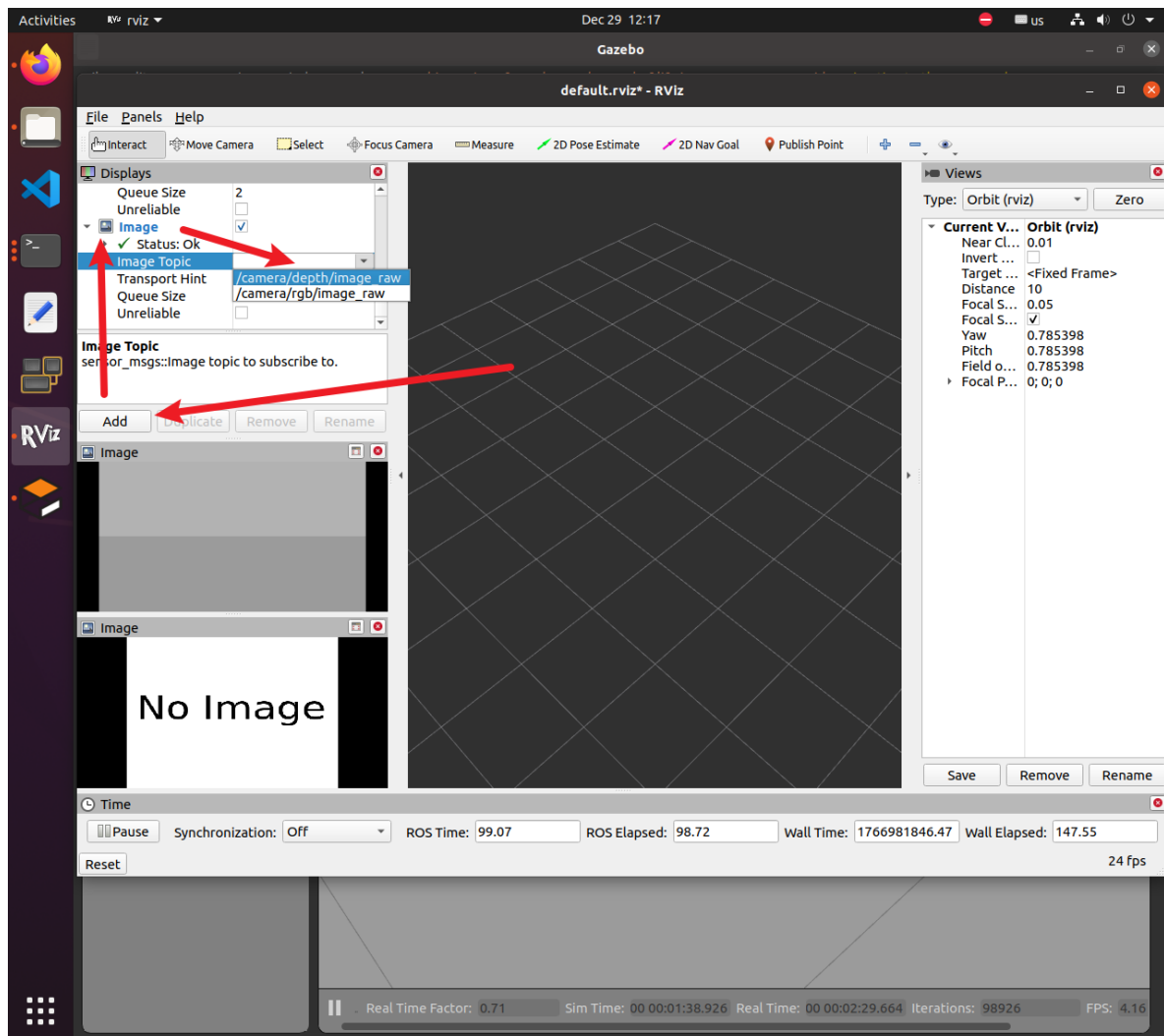
Set the topic to `/camera/rgb/image_raw`



Add another Image display and set the topic to /camera/depth/image_raw

Adjust the Fixed Frame (e.g., camera_link or base_link) if needed

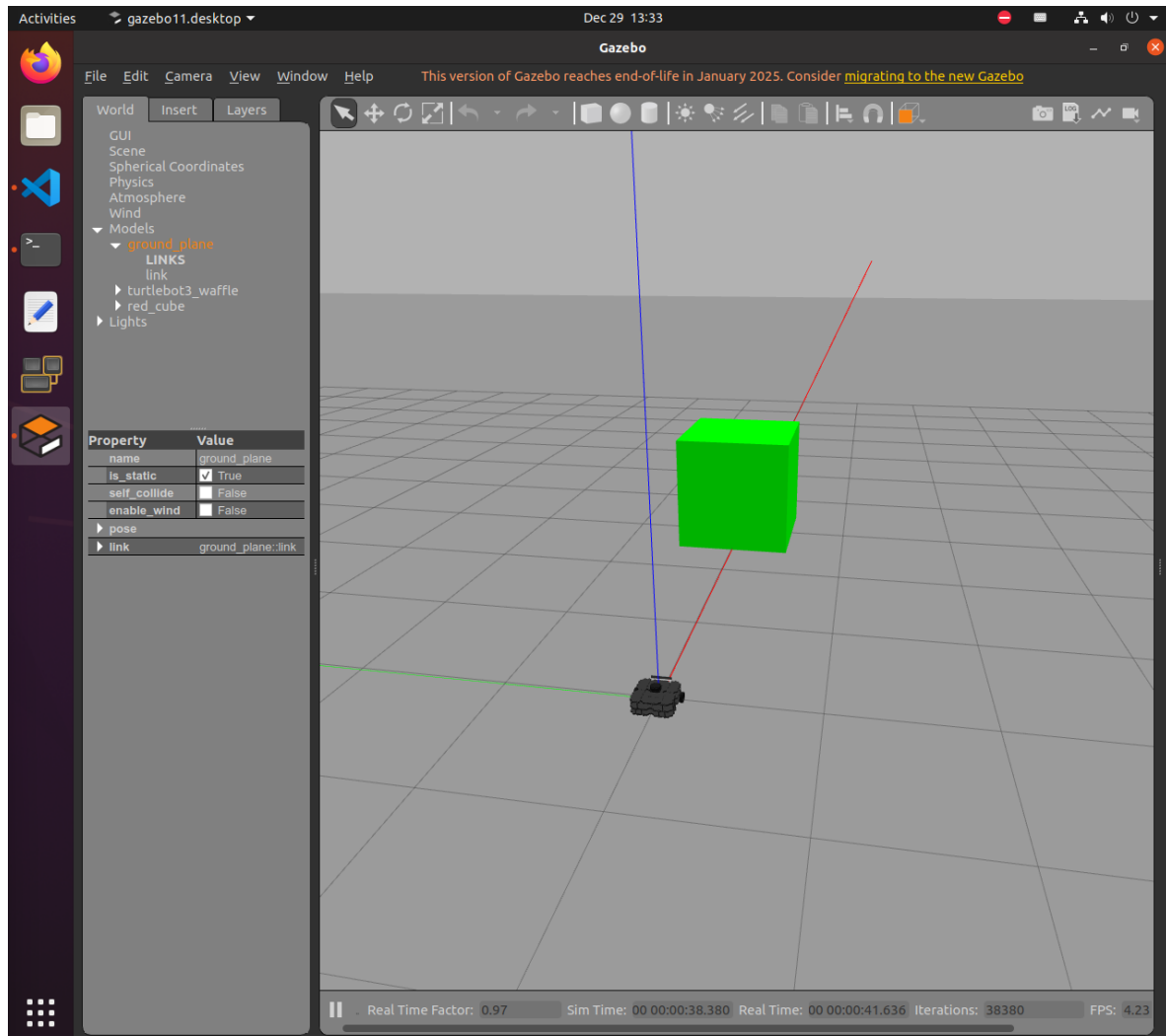
This allows you to visually inspect both the RGB stream and the depth map in real time.



1.3 Spawn an obstacle/object for detection

Set a green cube for object detection:

```
source ~/catkin_ws/devel/setup.bash
roslaunch gazebo_ros spawn_model \
  -file ~/catkin_ws/src/lab2_perception/demo_images/green_cube.sdf \
  -sdf \
  -model green_cube
```



Part 2: Image Processing Tools (HSV Tuning)

To detect a specific object (like a green block), we need to find the correct HSV (Hue, Saturation, Value) thresholds. We have provided tools to help you with this.

2.1 Generate Test Images

First, let's generate some sample images to test our algorithm.

```
# Generate dummy images in src/lab2_perception/demo_images/
roslaunch lab2_perception generate_images.py
```

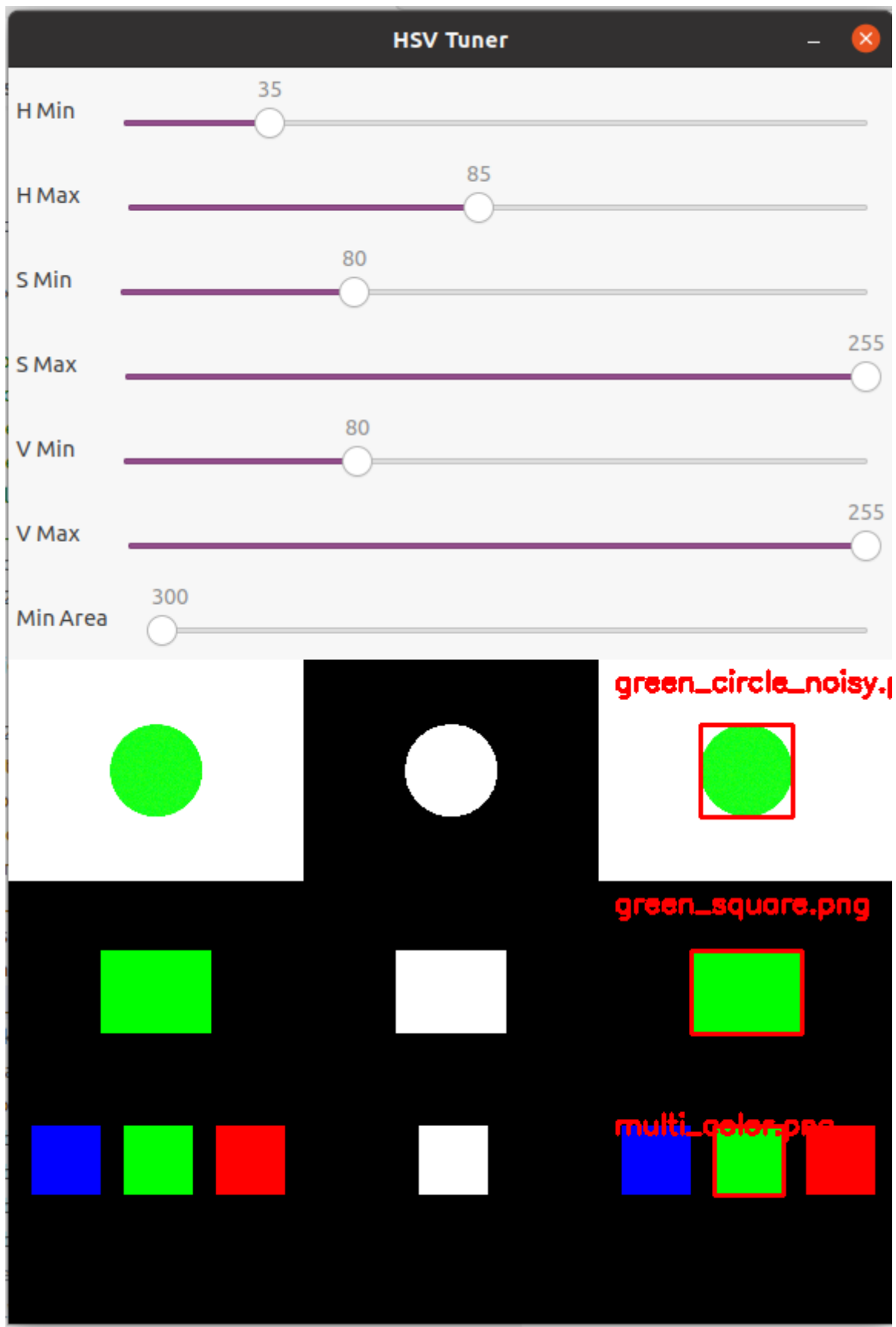
2.2 Tune HSV Thresholds

Use the tuner tool to find the best values to isolate the green color.

```
# Run the tuner tool
roslaunch lab2_perception hsv_tuner.py
```

Instructions:

- Adjust `H Min`, `S Min`, `V Min`, etc., until only the desired object is white in the middle mask window.
- `Esc` will exit the program.
- **Record these values.** You will need them for the main perception node. (Default values for green are already set in the code)



2.3 Get camera param

```
rostopic echo /camera/rgb/camera_info
```

```
s@ubuntu:~$ rostopic echo /camera/rgb/camera_info
header:
  seq: 11
  stamp:
    secs: 160
    nsecs: 752000000
  frame_id: "camera_rgb_optical_frame"
height: 1080
width: 1920
distortion_model: "plumb_bob"
D: [0.0, 0.0, 0.0, 0.0, 0.0]
K: [1206.8897719532354, 0.0, 960.5, 0.0, 1206.8897719532354, 540.5, 0.0,
    0.0, 1.0]
R: [1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0]
P: [1206.8897719532354, 0.0, 960.5, -84.48228403672648, 0.0, 1206.889771
    9532354, 540.5, 0.0, 0.0, 0.0, 1.0, 0.0]
binning_x: 0
binning_y: 0
roi:
  x_offset: 0
  y_offset: 0
  height: 0
  width: 0
  do_rectify: False
---
```

This parameter is critical for calculating the correct object coordinates.

This parameter will be loaded autonomously in the perception node.

Part 3: Integrated Perception & Visual Servoing

This is the core of the experiment. We will run the TurtleBot3 simulation in Gazebo. The robot will use its camera to detect a green object, calculate its 3D position, and drive towards it.

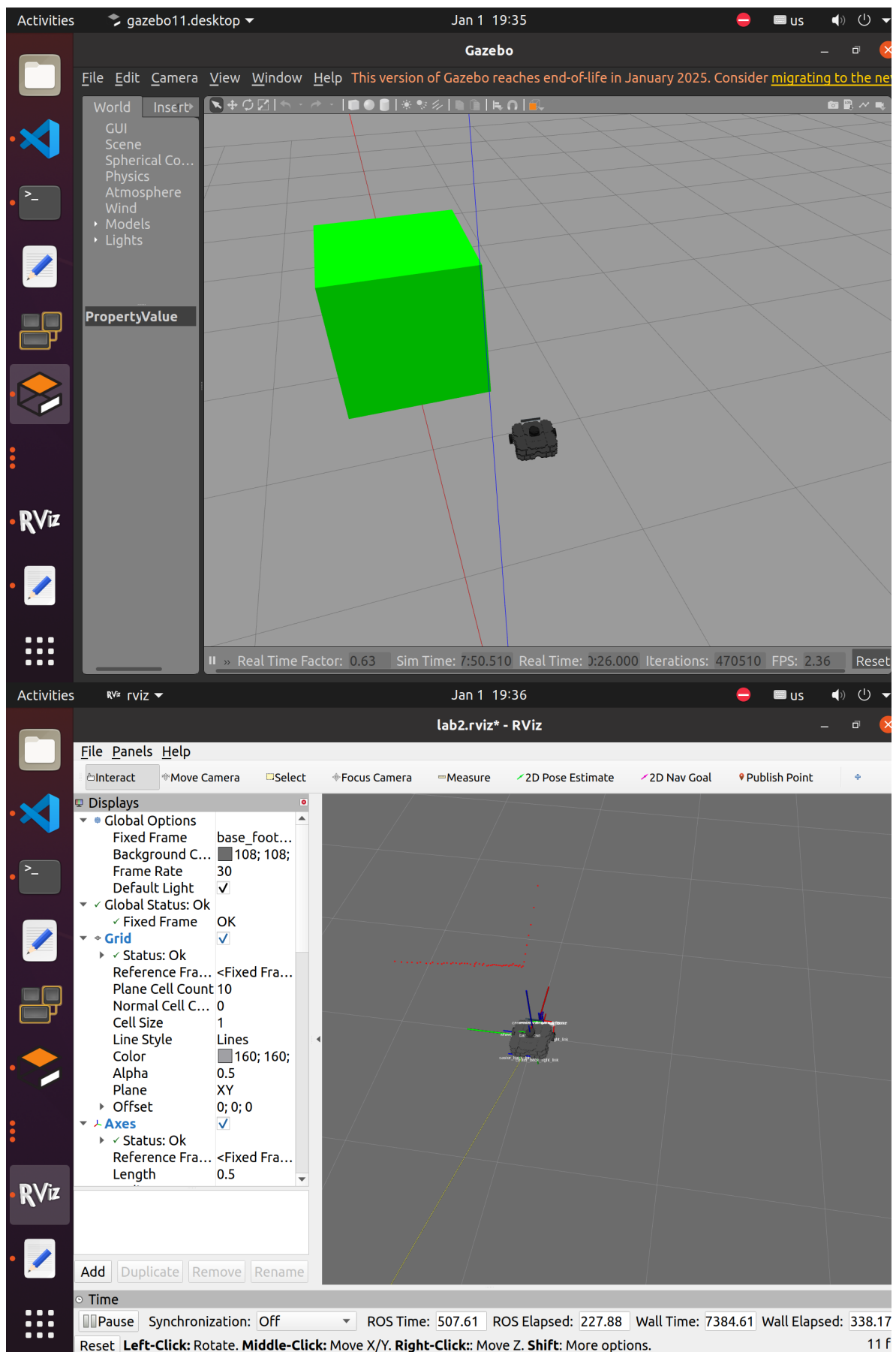
3.1 Launch Simulation and Perception Node

We have prepared a launch file that starts:

- Gazebo with TurtleBot3.
- The `perception_node` (performs detection and control).
- RViz for visualization.

```
# Close previous terminals if needed, then run:
# Terminal 1 start simulation
source ~/catkin_ws/devel/setup.bash
export TURTLEBOT3_MODEL=waffle
roslaunch turtlebot3_gazebo turtlebot3_empty_world.launch
```

```
# Terminal 2 start perception node
source ~/catkin_ws/devel/setup.bash
roslaunch lab2_perception lab2.launch
```

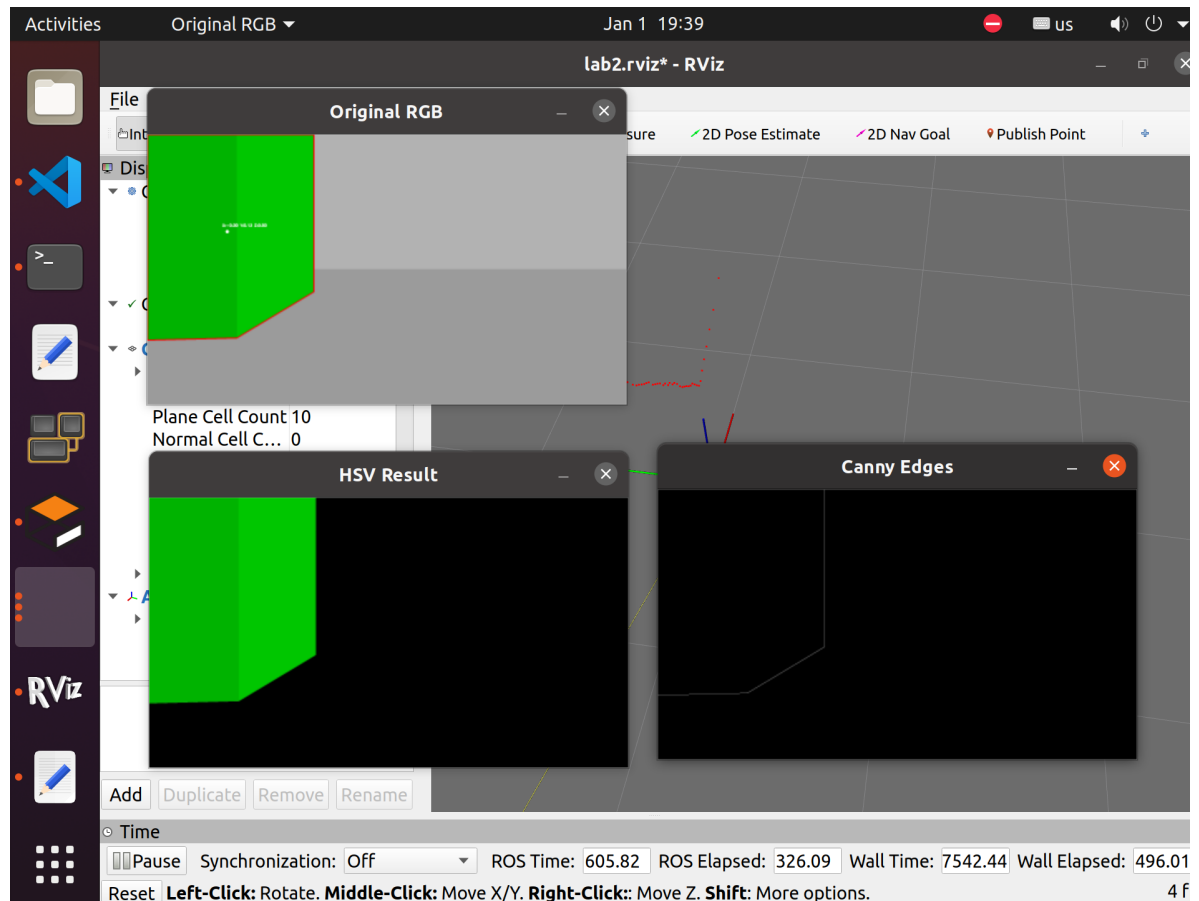
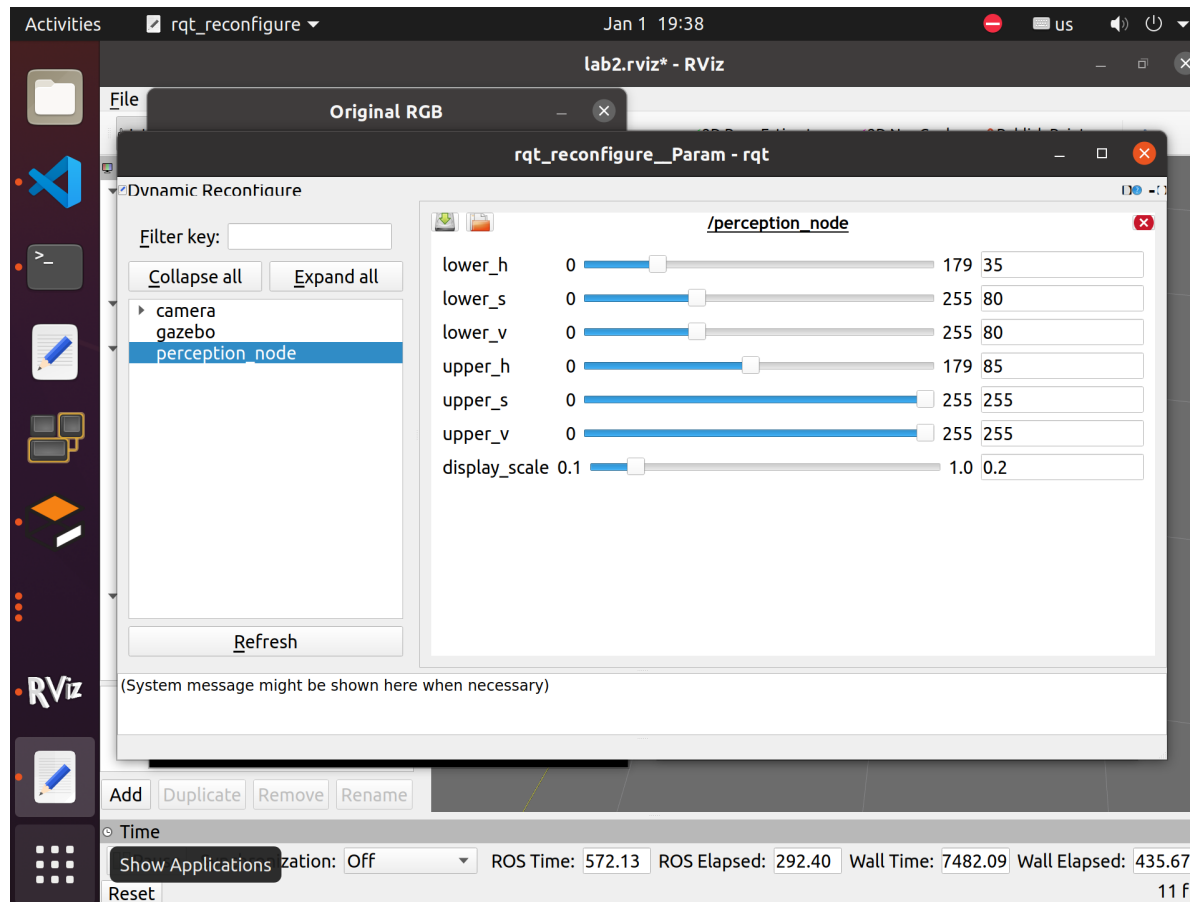



To stop the robot immediately, press Ctrl+C in the terminal running the perception node, or lift the robot (if real) / reset simulation (if Gazebo).

Use the HSV values (H_min, S_min, V_min, etc.) you recorded in Part 2 to update the parameters in the `rqt_reconfigure` window.

```
roslaunch rqt_reconfigure rqt_reconfigure
```

Set the threshold like this:



3.2 Data Visualization in Terminal

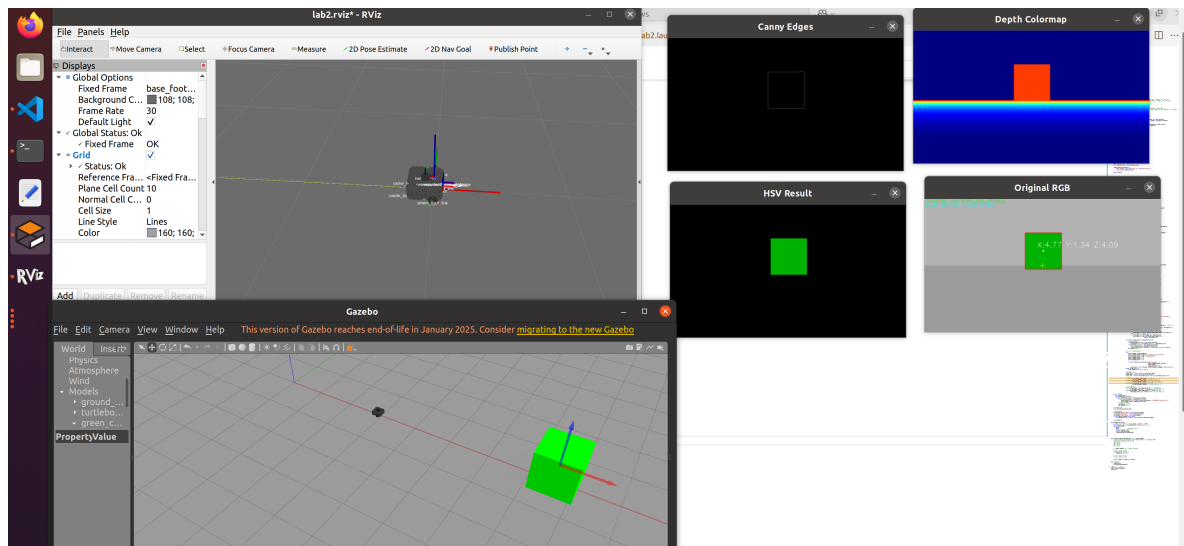
To see the custom messages being published:

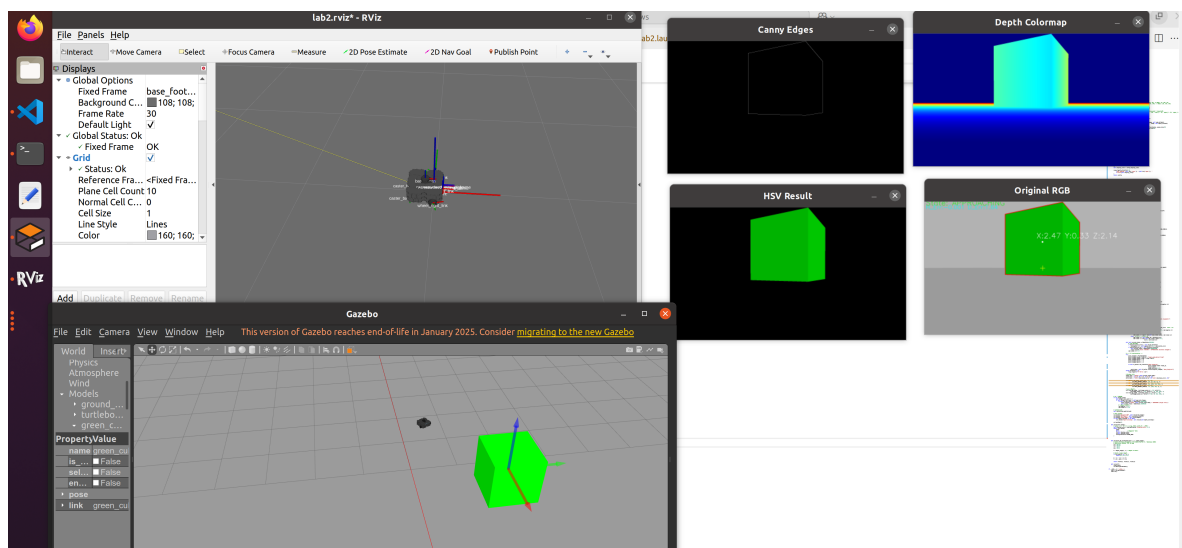
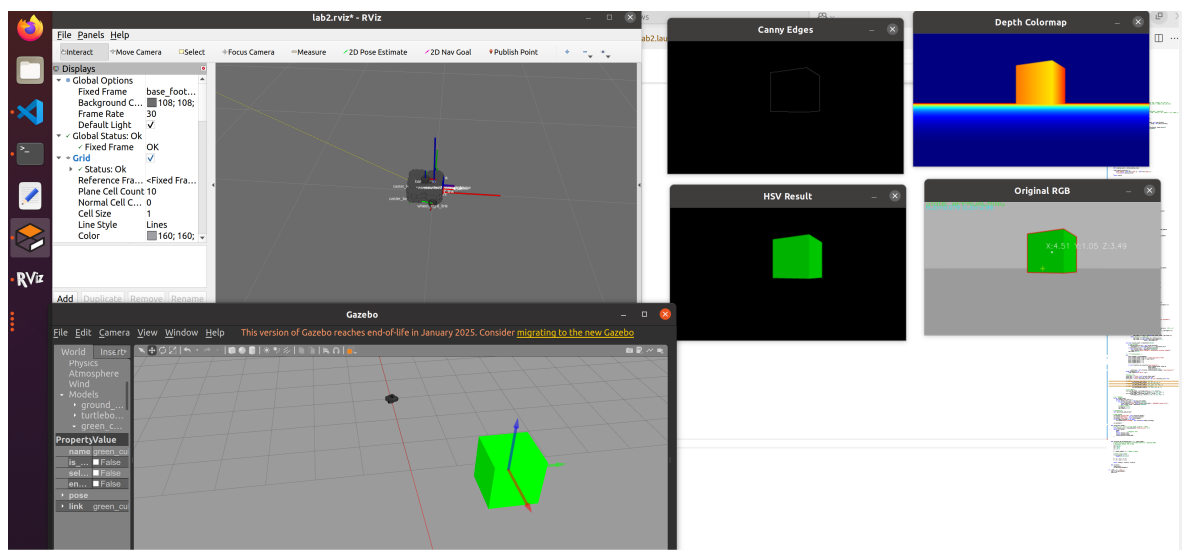
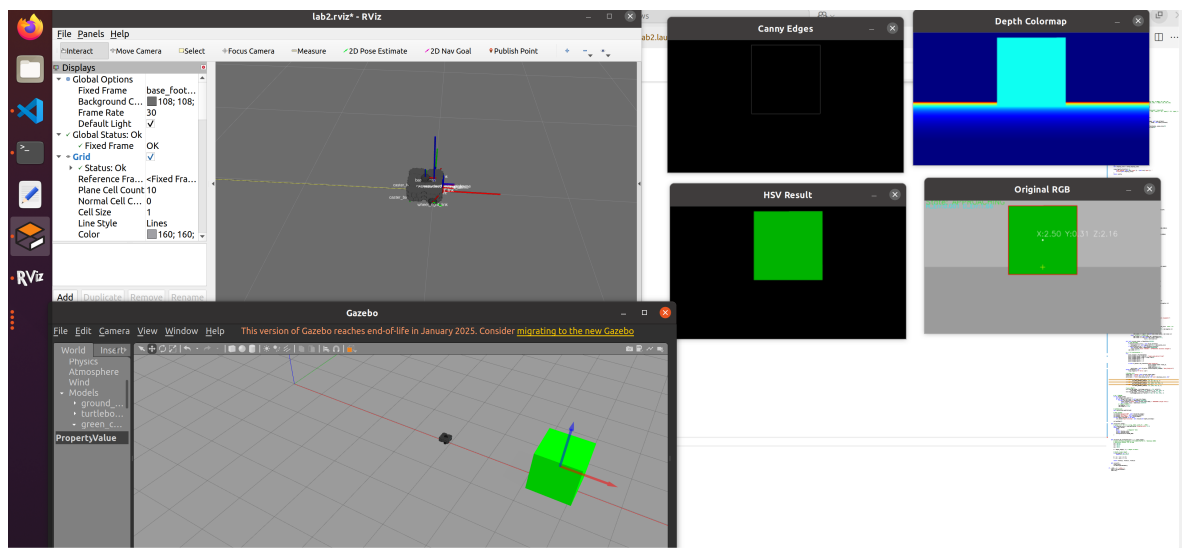
```
# Terminal 3
source ~/catkin_ws/devel/setup.bash

# Check the detected object coordinates
rostopic echo /detected_object
```

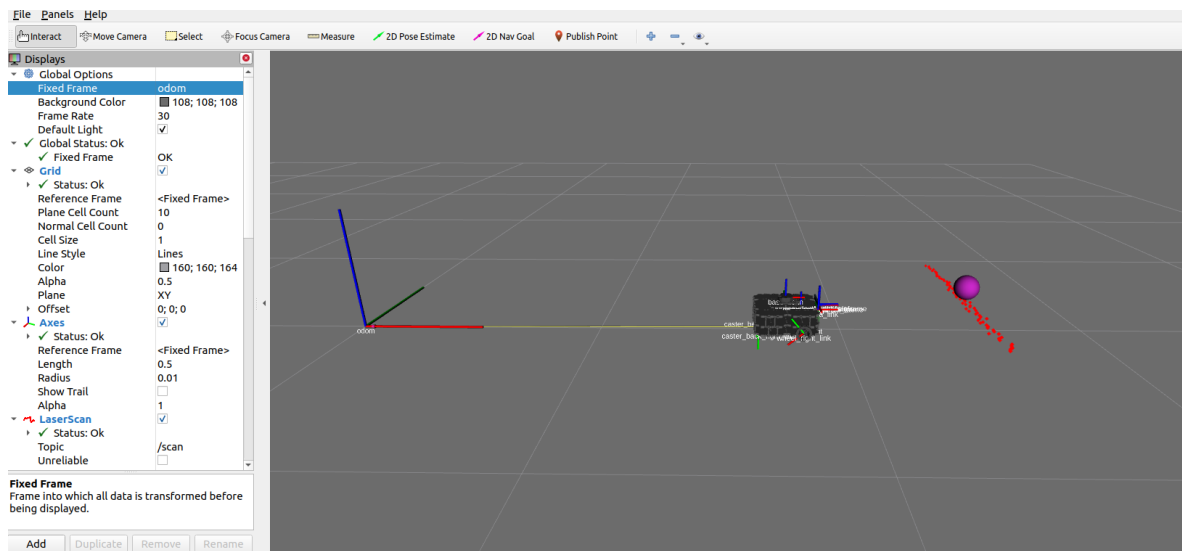
```
s@ubuntu:~$ source ~/catkin_ws/devel/setup.bash
s@ubuntu:~$ rostopic echo /detected_object
x: -0.0004510203143581748
y: 0.13305099308490753
z: 0.49995601177215576
---
x: -0.00045101967407390475
y: 0.133050799369812
z: 0.49995529651641846
---
x: -0.00045101993600837886
y: 0.13305087387561798
z: 0.49995559453964233
---
x: -0.0004510192957241088
y: 0.13305069506168365
z: 0.49995487928390503
---
x: -0.0004510189755819738
y: 0.1330505907535553
z: 0.4999545216560364
```

Move the cube, and you will observe the robot moving towards it.





In RViz, the center of mass of the detected object (the pink sphere) can be seen.



Check the velocity commands being sent to the robot
 rostopic echo /cmd_vel

```
angular:
  x: 0.0
  y: 0.0
  z: 0.017591339648173207
---
linear:
  x: 0.2
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.017591339648173207
---
linear:
  x: 0.2
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.017591339648173207
---
```

Part 5: Recording and Playback with rosbag

In this part, we will learn how to use **rosbag** to record and replay perception data. This allows us to debug and test perception algorithms offline without requiring a real robot or sensors.

5.1 Record Sensor Data in Simulation

First, launch the TurtleBot3 simulation environment.

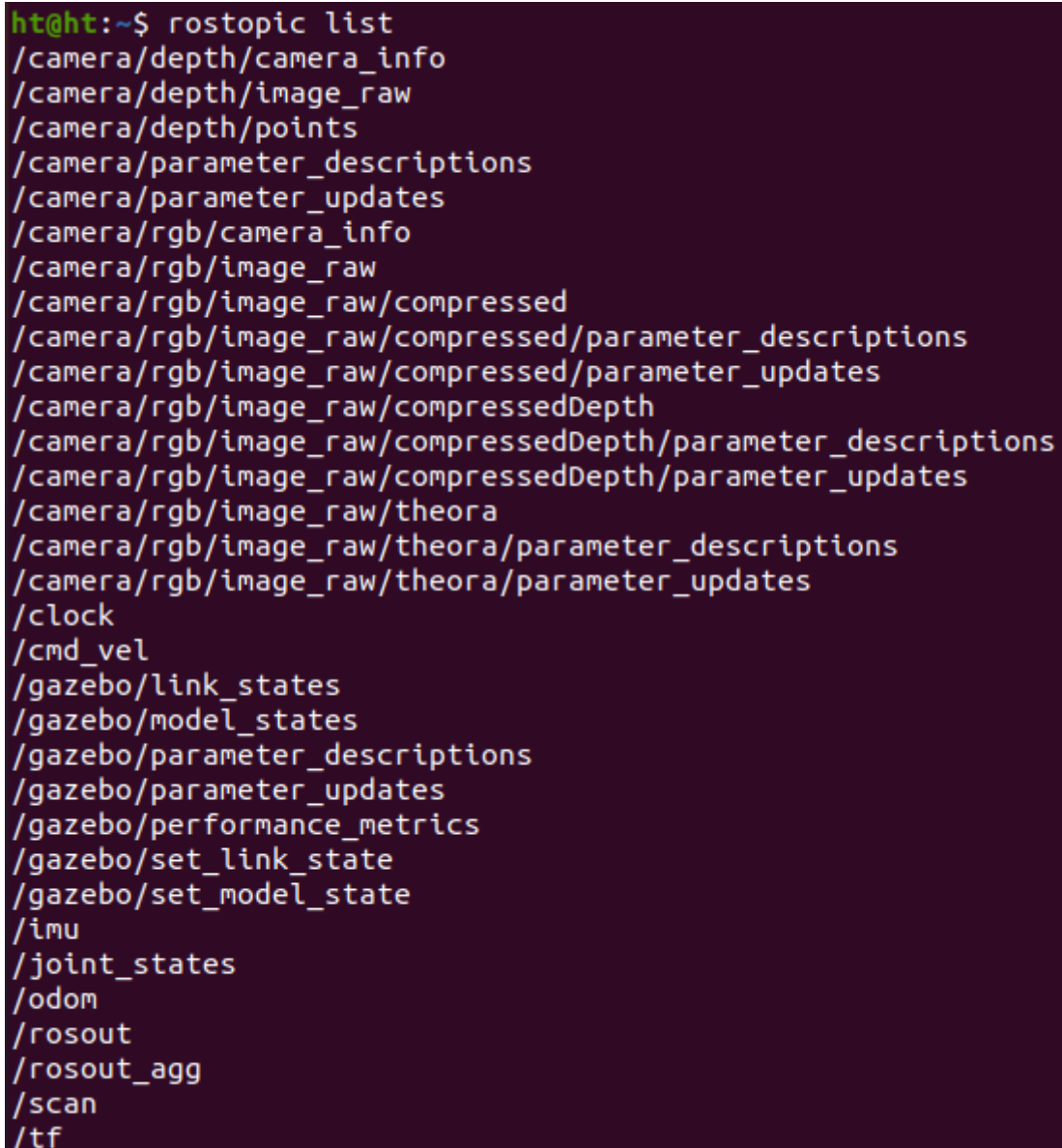
```
# Source workspace
source ~/catkin_ws/devel/setup.bash
export TURTLEBOT3_MODEL=waffle

# Launch simulation
roslaunch turtlebot3_gazebo turtlebot3_world.launch
```

Open a new terminal and inspect the available topics:

```
# List active topics
rostopic list
```

Identify camera-related topics such as RGB and depth images.



```
ht@ht:~$ rostopic list
/camera/depth/camera_info
/camera/depth/image_raw
/camera/depth/points
/camera/parameter_descriptions
/camera/parameter_updates
/camera/rgb/camera_info
/camera/rgb/image_raw
/camera/rgb/image_raw/compressed
/camera/rgb/image_raw/compressed/parameter_descriptions
/camera/rgb/image_raw/compressed/parameter_updates
/camera/rgb/image_raw/compressedDepth
/camera/rgb/image_raw/compressedDepth/parameter_descriptions
/camera/rgb/image_raw/compressedDepth/parameter_updates
/camera/rgb/image_raw/theora
/camera/rgb/image_raw/theora/parameter_descriptions
/camera/rgb/image_raw/theora/parameter_updates
/clock
/cmd_vel
/gazebo/link_states
/gazebo/model_states
/gazebo/parameter_descriptions
/gazebo/parameter_updates
/gazebo/performance_metrics
/gazebo/set_link_state
/gazebo/set_model_state
/imu
/joint_states
/odom
/rosout
/rosout_agg
/scan
/tf
```

5.2 Record Camera Data Using rosbag

Record the camera topics into a rosbag file.

```
# Record RGB and depth images
rosbag record -O camera_data.bag \
/camera/rgb/image_raw \
/camera/depth/image_raw
```

```
ht@ht:~$ rosbag record -o camera_data.bag /camera/rgb/image_raw /camera/depth/image_raw
[INFO] [1767706280.529847984]: Subscribing to /camera/depth/image_raw
[INFO] [1767706280.532205232]: Subscribing to /camera/rgb/image_raw
[INFO] [1767706280.750864353, 529.607000000]: Recording to 'camera_data.bag'.
```

- Let the simulation run for a short period.
- Press **Ctrl + C** to stop recording.

5.3 Replay Recorded rosbag Data

Stop the simulation, then replay the recorded data.

```
# Replay recorded data
rosbag play camera_data.bag
```

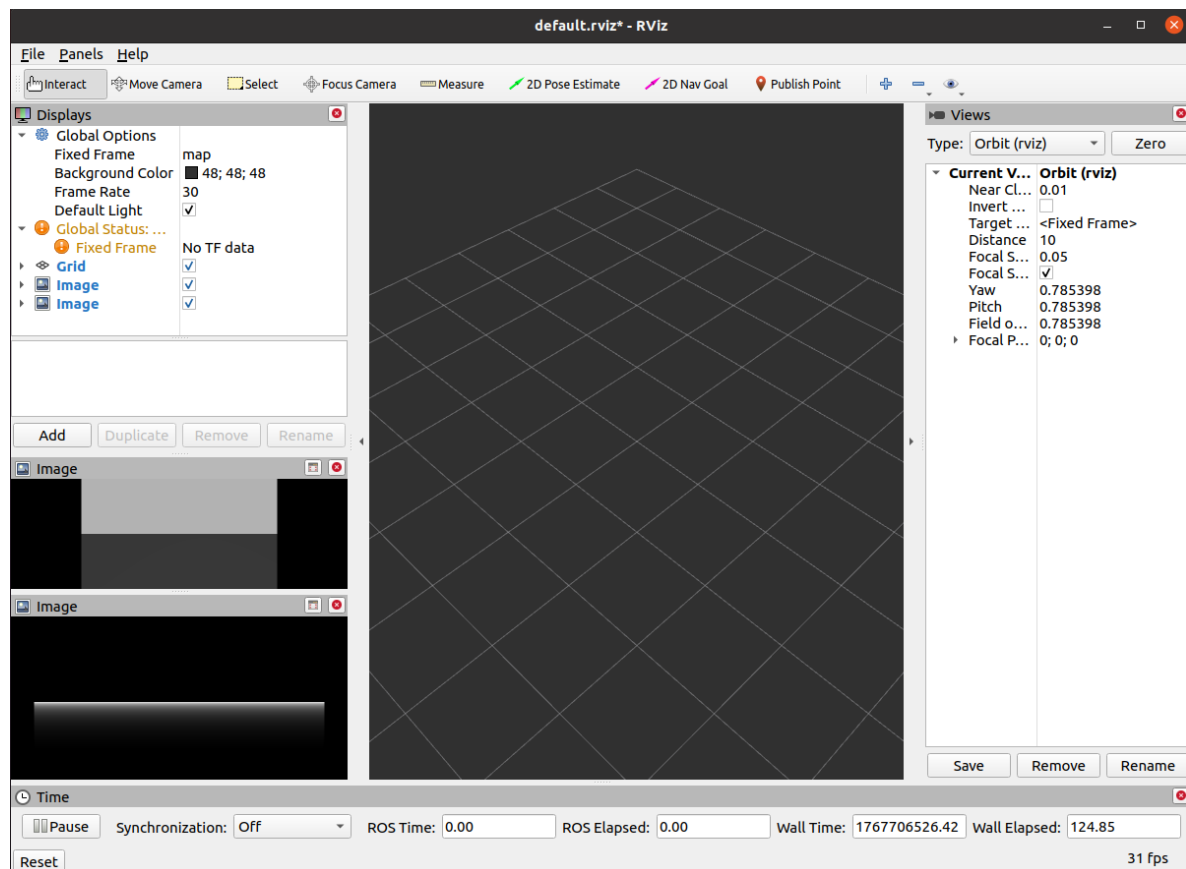
- The recorded sensor data will now be published as if the robot were running live.

5.4 Visualize Replayed Data in RViz

While the rosbag is playing, open RViz in a new terminal.

```
# Start RViz
rviz
```

- Add an **Image** display.
- Select `/camera/rgb/image_raw` or `/camera/depth/image_raw`.
- You should see the replayed camera data.



5.5 Replay Real-World Dataset (KITTI)

Next, replay a rosbag recorded in a real environment.

```
# Play real-world rosbag
roslaunch kitti_play kitti_bag.launch
```

The `kitti.bag` file can be downloaded using **one of the following methods**

(It has already been downloaded to the public classroom computers at `~/data/kitti.bag`):

1. **OneDrive**

https://cuhko365-my.sharepoint.com/:u:/g/personal/225080006_link_cuhk_edu_cn/IQCeVx_F2WMQZk08VXyADd5ATyv4ioLNLgMc3Vw3wgvFBk?e=ydcRxf

2. **Baidu Netdisk**

Link: <https://pan.baidu.com/s/1u41ODlrhgEzJCPwf5vBEew?pwd=vqa8>

Extraction code: `vqa8`

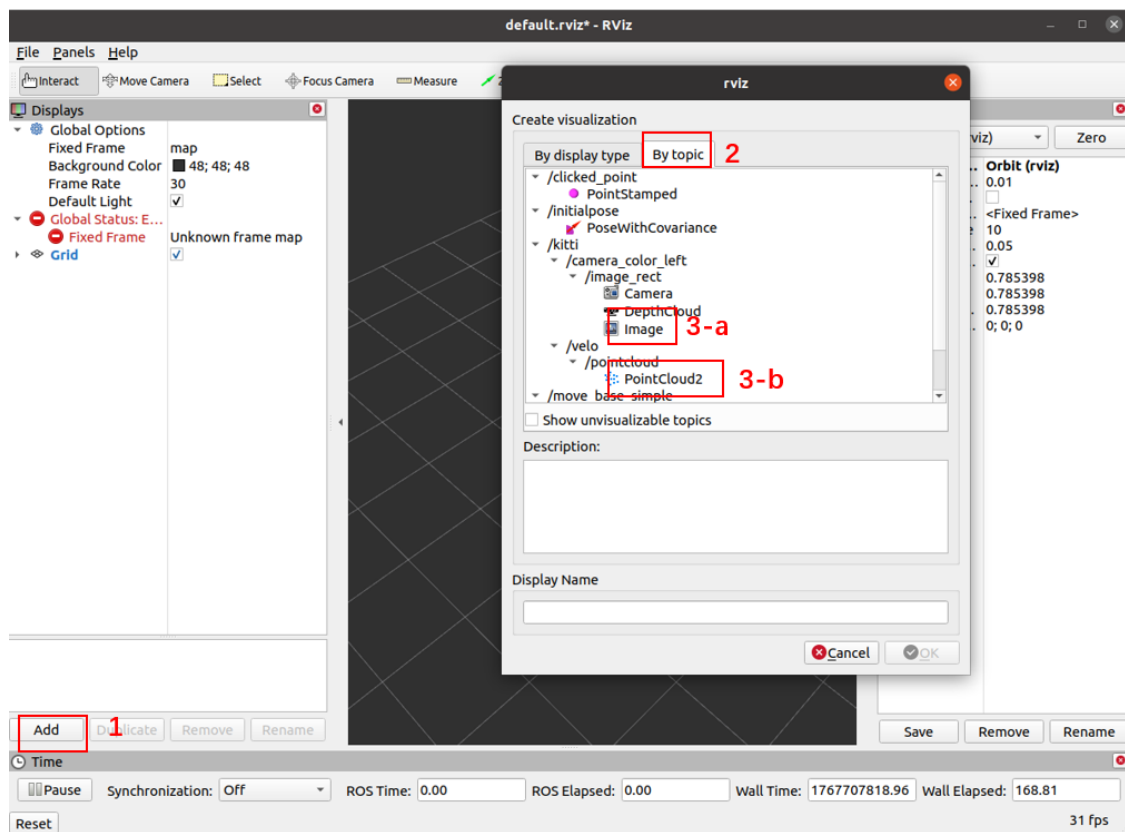
5.6 Visualize Camera and LiDAR Data

While the rosbag is playing:

1. Open RViz

2. Add displays for:

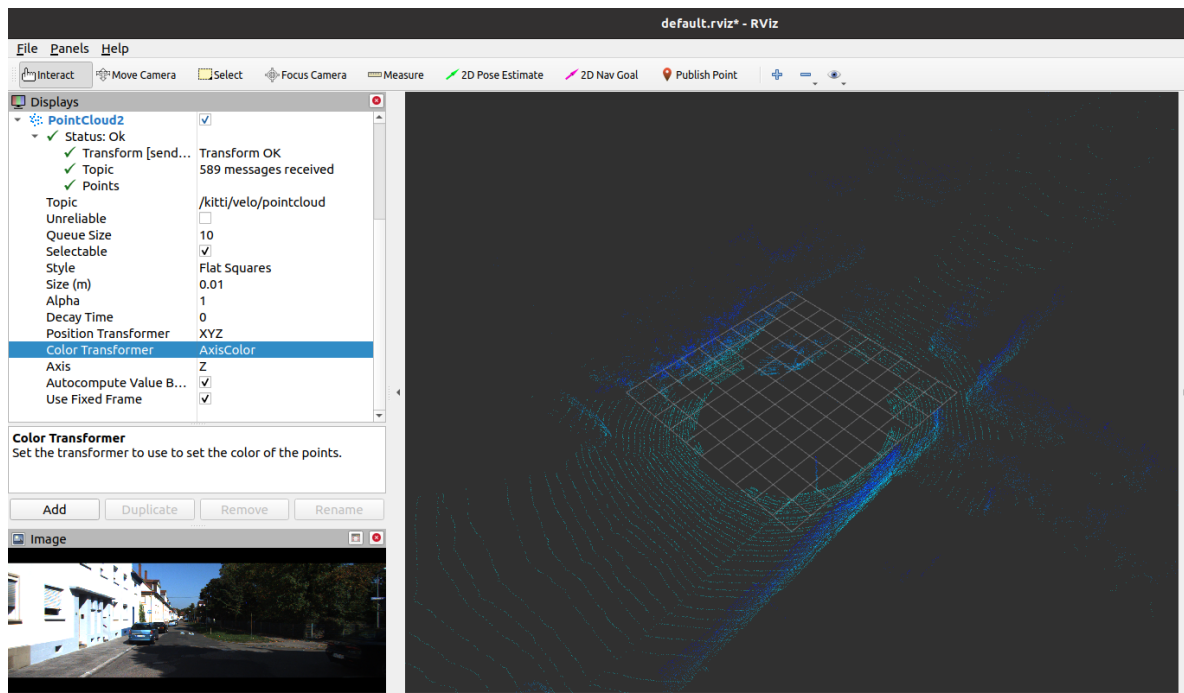
- Camera image (1->2->3-a)
- PointCloud2 (LiDAR) (1->2->3-b)



3. Set the **Fixed Frame** to:

```
velodyne
```

- You should now see both camera images and LiDAR point clouds.
- This demonstrates multi-sensor perception using recorded data.



Conclusion:

In this experiment, we successfully implemented a complete ROS perception pipeline for a mobile robot.

Data Acquisition: We learned how to subscribe to and visualize RGB and Depth data streams from the TurtleBot3 camera using RViz.

Image Processing: By utilizing the HSV color space and OpenCV tools, we effectively isolated specific objects (green cube) from the background and tuned thresholds for robust detection.

3D Localization & Control: We bridged the gap between 2D image pixels and 3D world coordinates using camera intrinsics and depth maps. Through TF transformations, we converted these coordinates into the robot's base frame to implement a closed-loop visual servoing controller, allowing the robot to autonomously track the target.

Visualization: Finally, we explored 3D scene reconstruction using Open3D, verifying the depth data fidelity.

This lab demonstrates the fundamental workflow of robot perception: Sense (Camera) → Perceive (OpenCV/HSV) → Plan/Act (Visual Servoing). These skills form the foundation for more advanced tasks such as SLAM, object manipulation, and semantic scene understanding.